Hybrid RAG Implementation in WebletGPT

This document explains how the Retrieval-Augmented Generation (RAG) knowledge base is implemented in WebletGPT. It uses a **Hybrid Search** approach, combining vector similarity with PostgreSQL full-text search, and merges the results using Reciprocal Rank Fusion (RRF) — closely mirroring how OpenAI Custom GPTs and OpenClaw work.

🏗 Architecture

The RAG system consists of two main pipelines: **Data Ingestion** (when a user uploads a file) and **Data Retrieval** (when the user asks a question in the chat).

1. Data Ingestion Pipeline (`/api/weblets/[id]/knowledge`)

When a user uploads a file in the Builder UI Knowledge Tab, it goes through the `processKnowledgeFile` pipeline (`lib/knowledge/process.ts`):

1. **Extraction (`lib/knowledge/extract.ts`)**
   - **PDF/DOCX**: Sent to the LlamaParse API to extract structured text accurately from complex layouts.
   - **MD/TXT/CSV**: Read directly as raw text.
2. **Chunking (`lib/knowledge/chunk.ts`)**
   - We use a **Markdown-Aware Smart Chunker**.
   - Instead of blindly cutting every 2000 characters (which can break sentences), the chunker attempts to split by:
     1. Markdown Headers (`## Section 1`)
     2. Paragraphs (double newlines)
     3. Sentences (if paragraphs are too long)
   - This ensures each chunk contains a coherent legal section or topic. Includes a 50-token overlap to maintain context between chunks.
3. **Embedding (`lib/knowledge/embed.ts`)**
   - Supports dual providers (configured via `.env`):
     - **Ollama**: Uses `nomic-embed-text-v2-moe` (Free, Local, 768 dimensions)
     - **OpenAI**: Uses `text-embedding-3-small` (Paid, Cloud, 1536 dimensions)
   - Converts the text chunks into mathematical vectors (arrays of floating-point numbers).
4. **Storage (`prisma/schema.prisma`)**
   - The chunks and their vector embeddings are stored in the PostgreSQL (Neon) database in the `KnowledgeChunk` table.
   - The `embedding` column uses the `vector` type provided by the `pgvector` extension.

2. Data Retrieval Pipeline (`lib/tools/file-search.ts`)

During chat, if the AI needs information, it invokes the `fileSearchTool`. This tool performs a **Hybrid Search**:

1. **Semantic Vector Search (pgvector)**
   - The user's query is converted into a vector using the same embedding model.

    - Neon database performs a Cosine Similarity search (`<=>`) to find chunks with the closest meaning, even if the exact keywords don't match (e.g., "broken promise" finds "breach of contract").
2. **Full-Text Keyword Search (PostgreSQL `tsvector`)**
    - In parallel, the query is converted into a `tsquery` (e.g., `breach & contract`).
    - The database performs a traditional keyword search against the `content` column using PostgreSQL's `to_tsvector` and ranks them using `ts_rank`. This ensures exact terminology matching.
3. **Reciprocal Rank Fusion (RRF)**
    - The results from both searches are combined using the RRF algorithm.
    - A chunk that ranks #1 in Vector Search and #2 in Full-Text Search gets a massive boost, bubbling to the top. This provides the ultimate balance of context (vector) and accuracy (keyword).

🚶 Walkthrough Example: "Pakistan Contract Law"

Here is a step-by-step example of how the system handles a real document and question.

Phase 1: Uploading the Document

1. **User Action**: You drag and drop `test-docs/pakistan-contract-law.md` into the Knowledge Tab.
2. **Extraction**: The system reads the Markdown file containing chapters on formation, breach, remedies, etc.
3. **Chunking**: Because it's markdown, `chunk.ts` splits it intelligently:
    - **Chunk 1**: `Chapter 1: Formation of Contracts (Sections 2-14)`
    - **Chunk 2**: `Chapter 2: Breach of Contract (Sections 73-75)`
    - **Chunk 3**: `Chapter 3 & 4: Types of Breach & Remedies`
4. **Embedding**: Ollama converts each of those 3 chunks into 768-dimensional vectors.
5. **Storage**: The 3 chunks are saved to the `KnowledgeChunk` table in Neon, linked to your Weblet's ID.

Phase 2: Asking a Question

1. **User Action**: You ask the bot: "*What happens if someone breaks a contract?*"
2. **Tool Invocation**: The AI decides to use the `fileSearchTool` with the query: `"consequences of breaking a contract Pakistan"`.
3. **Query Embedding**: The query is converted into a vector by Ollama.
4. **Hybrid Search Execution**:
    - **Vector Search**: Finds Chunk 2 (`Chapter 2: Breach of Contract`) because "breaking" is semantically similar to "breach".
    - **Keyword Search**: Looks for the words `consequences & break & contract`.
5. **RRF Merge**: Chunk 2 is ranked highest by both methods and is returned as the top result.
6. **LLM Synthesis**: The raw text of Chunk 2 (mentioning Section 73, Compensatory Damages) is passed back to the LLM.
7. **Final Answer**: The AI reads the chunk and responds to you:

> "*Under Section 73 of the Pakistan Contract Act, if someone breaks a contract, the suffering party is entitled to receive compensation for losses that naturally arose from the breach...*"

⚙️ Requirements & Key Files

- **Database**: PostgreSQL with `pgvector` extension enabled (Neon provides this out of the box).
- **Env Vars**:
  - `EMBEDDING_PROVIDER` (set to `ollama` or `openai`)
  - `LLAMA_CLOUD_API_KEY` (if extracting PDFs)
- **Key Files to Understand**:
  - `lib/knowledge/process.ts` (The orchestrator)
  - `lib/knowledge/chunk.ts` (Smart Markdown Chunker)
  - `lib/knowledge/embed.ts` (Dual-provider embeddings)
  - `lib/tools/file-search.ts` (The Hybrid RAG + RRF search implementation)